

FASTER: Fast and Safe Trajectory Planner For Unknown Dynamic Environment

Unni Aditya Mohan
Khoury College of Computer Science
Northeastern University
Boston, USA
lnu.un@northeastern.edu

Anurag Girish Kadkol
Khoury College of Computer Science
Northeastern University
Boston, USA
kadkol.a@northeastern.edu

Abstract—High-speed autonomous navigation through unknown environments requires trajectory planners that are both fast and safe. FASTER (Fast and Safe Trajectory Planner) addresses this challenge by planning simultaneously in free-known and unknown space while maintaining a safe backup trajectory entirely within the free-known region. However, FASTER assumes a fully static world, which limits its applicability to real-world scenarios involving moving obstacles. In this work, we first conduct a partial reproducibility study of FASTER in static environments, evaluating its performance in 2-D (modeled in python) and 3-D environments (modeled in ROS/Gazebo). We then propose FASTER+Predict, an extension that incorporates moving obstacles by augmenting the occupancy and unknown grids with time-varying inflation regions derived from a constant-velocity prediction model, enabling FASTER’s existing MIQP to plan around dynamic obstacles without modifying the optimization formulation itself.

I. INTRODUCTION

Autonomous navigation at high speed through unknown, cluttered environments is a fundamental challenge in mobile robotics. The core difficulty lies in a tradeoff between safety and speed: a planner must generate collision-free, dynamically feasible trajectories in real time while only having partial knowledge of the surrounding environment. Conservative planners that restrict all motion to the free-known space $\mathcal{F}$ and impose a terminal stopping condition can guarantee safety, but the braking maneuver propagates backward through the trajectory, severely limiting the achievable velocity—especially when $\mathcal{F}$ is small relative to the total environment.

Tordesillas et al. [1] introduced FASTER (Fast and Safe Trajectory Planner), an optimization-based framework that resolves this tradeoff. The key insight is to solve two trajectory optimization problems at each replanning step: a *Whole Trajectory* optimized over both $\mathcal{F}$ and the unknown space $\mathcal{U}$, which provides a long planning horizon and high nominal speed, and a *Safe Trajectory* contained entirely in $\mathcal{F}$, which serves as a guaranteed fallback. The robot commits only to the initial segment of the Whole Trajectory (up to a carefully chosen point R), concatenated with the Safe Trajectory. This committed trajectory is provably collision-free under a static-world assumption, while the Whole Trajectory’s optimistic use

of unknown space enables speeds up to 7.8 m/s in simulation—roughly twice the velocity achieved by prior methods.

We selected FASTER for this project for three reasons. First, it addresses a fundamental open problem at the intersection of trajectory optimization and planning under partial observability, both core topics in mobile robotics. Second, the algorithm has a clear mathematical formulation—a Mixed-Integer Quadratic Program (MIQP) with Bézier curve containment constraints—that lends itself to rigorous empirical evaluation. Third, the authors provide open-source code and detailed simulation benchmarks, enabling meaningful reproducibility analysis.

Despite its strengths, FASTER makes a critical assumption: the environment is fully static. Theorem 1 in [1], which guarantees that every committed trajectory lies within free or free-unknown space, relies explicitly on the invariance condition $\mathcal{F}_k \cup \mathcal{F}_{\text{Unknown},k} = \mathcal{F}_{k+1} \cup \mathcal{F}_{\text{Unknown},k+1}$ —that is, space classified as free never becomes occupied. In any environment with moving obstacles, this assumption is violated: a region that was free at time t may become occupied at $t + \Delta t$, potentially invalidating the committed trajectory mid-execution.

This limitation motivates the central contribution of our project. We first reproduce the key simulation experiments from [1] to verify the algorithm’s claimed performance in static settings. We then propose *FASTER+Predict*, an extension that incorporates moving obstacles into the planning framework by:

- 1) Predicting obstacle positions forward in time using a constant-velocity model.
- 2) Inflating predicted obstacle volumes by a safety margin that accounts for prediction uncertainty.
- 3) Augmenting the MIQP constraints at each replanning step to exclude time-varying obstacle regions from the convex corridor.

We evaluate this extension quantitatively, measuring success rate, flight time, and average velocity as functions of the number of dynamic obstacles M and their speed $\|\dot{q}_i\|$.

II. PROBLEM STATEMENT

We consider a robot navigating through a three-dimensional workspace. The workspace is partitioned as $\mathbb{R}^3 = \mathcal{F} \cup \mathcal{O} \cup \mathcal{U}$,

where $\mathcal{F}$, $\mathcal{O}$, and $\mathcal{U}$ denote the free-known, occupied-known, and unknown regions, respectively. These sets are disjoint and updated online as the robot's onboard depth sensor reveals new portions of the environment.

A. Robot Dynamics

The robot is modeled as a triple integrator with state vector

$$\mathbf{x} = [\mathbf{p}^\top \quad \mathbf{v}^\top \quad \mathbf{a}^\top]^\top \in \mathbb{R}^9 \quad (1)$$

where $\mathbf{p}$, $\mathbf{v}$, and $\mathbf{a}$ are the position, velocity, and acceleration vectors, respectively. The control input is the jerk $\mathbf{u} = \mathbf{j} \in \mathbb{R}^3$. Dynamic feasibility is enforced through box constraints $\|\mathbf{v}\|_\infty \leq v_{\max}$, $\|\mathbf{a}\|_\infty \leq a_{\max}$, and $\|\mathbf{j}\|_\infty \leq j_{\max}$.

B. Trajectory Parameterization

The trajectory is parameterized as a spline of N cubic polynomial intervals, each of uniform duration d_t . On interval $n \in \{0, \dots, N-1\}$, the position is:

$$\mathbf{p}_n(\tau) = \mathbf{a}_n \tau^3 + \mathbf{b}_n \tau^2 + \mathbf{c}_n \tau + \mathbf{d}_n, \quad \tau \in [0, d_t] \quad (2)$$

Each polynomial interval is equivalently represented by four Bézier control points $\mathbf{r}_{nj}$ ($j = 0, \dots, 3$). Since a Bézier curve is contained within the convex hull of its control points, constraining all four control points of an interval to lie inside a convex polyhedron guarantees that the entire interval remains within that polyhedron.

C. MIQP Formulation

At each replanning step, the planner solves the following Mixed-Integer Quadratic Program over jerk inputs $\mathbf{j}_n$ and binary interval-to-polyhedron assignment variables $b_{np} \in \{0, 1\}$:

$$\min_{\mathbf{j}_n, b_{np}} \sum_{n=0}^{N-1} \|\mathbf{j}_n\|^2 d_t \quad (3)$$

subject to:

$$\mathbf{x}_0(0) = \mathbf{x}_{\text{init}}, \quad \mathbf{x}_{N-1}(d_t) = \mathbf{x}_{\text{final}} \quad (4)$$

$$\mathbf{x}_{n+1}(0) = \mathbf{x}_n(d_t), \quad n = 0, \dots, N-2 \quad (5)$$

$$\|\mathbf{v}_n(0)\|_\infty \leq v_{\max}, \quad \|\mathbf{a}_n(0)\|_\infty \leq a_{\max}, \quad \|\mathbf{j}_n\|_\infty \leq j_{\max} \quad (6)$$

$$b_{np} = 1 \implies A_p \mathbf{r}_{nj} \leq \mathbf{c}_p, \quad \forall j \quad (7)$$

$$\sum_{p=0}^{P-1} b_{np} \geq 1, \quad \forall n \quad (8)$$

where $\{(A_p, \mathbf{c}_p)\}_{p=0}^{P-1}$ are the half-space representations of P overlapping convex polyhedra obtained from a convex decomposition of the planning space. Constraint (7) ensures that if interval n is assigned to polyhedron p , all of its Bézier control points lie within that polyhedron. Constraint (8) requires each interval to be assigned to at least one polyhedron.

This MIQP is solved *twice* per replanning step: once for the Whole Trajectory in $\mathcal{F} \cup \mathcal{U}$ (providing speed), and once for the Safe Trajectory in $\mathcal{F}$ alone (providing safety). The committed trajectory is:

$$\pi_{\text{committed}} = \pi_{\text{whole}}^{A \rightarrow R} \cup \pi_{\text{safe}} \subset \mathcal{F} \quad (9)$$

where R is a point along the Whole Trajectory chosen such that the robot can safely transition to the backup.

D. Dynamic Obstacle Model

The original FASTER formulation assumes the world is static. We extend the problem to include M moving obstacles. Each obstacle i has an observed position $\mathbf{q}_i(t_0)$ and velocity $\dot{\mathbf{q}}_i(t_0)$ at the current time t_0 , obtained from the perception pipeline. The predicted position at a future time t is:

$$\hat{\mathbf{q}}_i(t) = \mathbf{q}_i(t_0) + \dot{\mathbf{q}}_i(t_0)(t - t_0) \quad (10)$$

Each predicted obstacle is inflated by a safety margin ϵ to account for prediction error, yielding a time-varying exclusion region $D_i(t)$.

The trajectory planning problem is now to find a collision-free, dynamically feasible trajectory that avoids $\mathcal{O}(t)$ for all t along the trajectory, while maintaining a safe backup in the (time-varying) free-known space.

III. PROPOSED SOLUTION

A. Reproducibility Evaluation of FASTER

We first reproduce a part of the core simulation experiments of FASTER within a ROS/Gazebo environment. The simulation world consists of a configurable three-dimensional obstacle field (random forests and structured environments). A simulated depth camera with a 90° horizontal field of view provides the perception input, which is fused into a body-centered sliding occupancy grid map using 3-D Bresenham ray-tracing. Jump Point Search (JPS) serves as the global planner, providing a shortest piece-wise linear path from the robot to the goal. The convex decomposition of the free space along this path produces the overlapping polyhedra used as corridor constraints in the MIQP.

For the reproducibility study, we evaluate the following metrics across multiple randomized environments:

- **Success rate:** fraction of trials in which the robot reaches the goal without collision.
- **Total flight time:** time from start to goal arrival.
- **Path Length:** Distance travelled from start to goal arrival.

B. FASTER+Predict Extension

We extend FASTER to handle dynamic obstacles by augmenting its occupancy and unknown grids with velocity- and distance-aware inflation bubbles, while leaving the MIQP formulation and trajectory planner itself untouched.

1) *Obstacle Detection and Velocity Estimation:* At each replanning step, candidate dynamic obstacles are identified from the onboard depth-camera point cloud. Points are raycast into voxels of resolution $r_v = 0.15$ m by the occupancy mapper. We restrict attention to a sensing region of radius $R_s = 10$ m around the drone; voxels outside R_s are treated as unknown. Within this region, occupied voxels are clustered into candidate obstacles and each cluster's centroid $\mathbf{p}_i(t)$ is tracked across consecutive frames. Translational velocity is estimated via finite differencing with exponential smoothing:

$$\hat{\mathbf{v}}_i(t) = \alpha \frac{\mathbf{p}_i(t) - \mathbf{p}_i(t - \Delta t)}{\Delta t} + (1 - \alpha) \hat{\mathbf{v}}_i(t - \Delta t), \quad (11)$$

with smoothing factor $\alpha = 0.7$.

To prevent bubble inflation from growing spuriously when the drone itself slows near an obstacle, we use an *effective* drone speed

$$v_d^*(t) = \max_{\tau \in [t-T_w, t]} \|\dot{\mathbf{q}}_d(\tau)\|, \quad (12)$$

i.e. the peak speed over a trailing window of $T_w = 2$ s, in place of the instantaneous speed.

2) *Two-Boundary Inflation Model*: The core of our extension is a two-boundary inflation scheme that maps directly onto FASTER's *occupancy* and *unknown* grids. For each tracked obstacle i with measured radius $\tilde{r}_i = r_i + \eta$, where $\eta \sim \mathcal{U}(-r_v, r_v)$ captures voxel quantization error, we define an *inner* and *outer* boundary:

$$\delta_i = \text{clip}\left(K \tilde{r}_i \frac{d_i}{v_d^* + \epsilon_v}, \delta_{\min}, \delta_{\max}\right), \quad (13)$$

$$R_i^{\text{in}} = \tilde{r}_i + \delta_i, \quad (14)$$

$$R_i^{\text{out}} = R_i^{\text{in}} + \Delta_s, \quad (15)$$

where $d_i = \|\mathbf{p}_i - \mathbf{q}_d\|_{xy}$ is the planar drone-obstacle distance, and the constants are $K = 0.2$, $\epsilon_v = 0.3$ m/s, $\delta_{\min} = 0.05$ m, $\delta_{\max} = 0.3$ m, and $\Delta_s = 0.4$ m.

Equation (13) has three design properties:

- $\delta_i \propto \tilde{r}_i$: larger obstacles command larger safety margins.
- $\delta_i \propto d_i$: bubbles grow when the obstacle is far, giving the planner time to commit to a detour well before the encounter.
- $\delta_i \propto 1/v_d^*$: bubbles shrink when the drone is moving fast near the obstacle, enabling the planner to thread gaps quickly rather than brake conservatively.

The regularizer ϵ_v prevents divergence as $v_d^* \rightarrow 0$, and the clips $[\delta_{\min}, \delta_{\max}]$ enforce a minimum safety envelope and prevent any single bubble from consuming the corridor.

3) *Occupancy-Unknown Grid Augmentation*: For each obstacle i we generate two disk-shaped voxel regions centered at $\mathbf{p}_i$:

$$\mathcal{O}_i^* = \{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x} - \mathbf{p}_i\| \leq R_i^{\text{in}}\}, \quad (16)$$

$$\mathcal{U}_i^* = \{\mathbf{x} \in \mathbb{R}^2 : R_i^{\text{in}} < \|\mathbf{x} - \mathbf{p}_i\| \leq R_i^{\text{out}}\}. \quad (17)$$

Each disk is discretized at the mapper resolution r_v and replicated across Z -levels spanning the drone's flight band. The augmented grids delivered to FASTER are:

$$\mathcal{O}_{\text{aug}} = \mathcal{O}_{\text{static}} \cup \bigcup_{i \in \mathcal{N}} \mathcal{O}_i^*, \quad \mathcal{U}_{\text{aug}} = \mathcal{U}_{\text{static}} \cup \bigcup_{i \in \mathcal{N}} \mathcal{U}_i^*, \quad (18)$$

where $\mathcal{N}$ is the set of the $N_{\text{near}} = 4$ closest obstacles within R_s . Two additional filters are applied for consistency: (i) voxels lying more than 0.5 m behind the drone in its heading frame are discarded, since the forward-facing depth camera cannot resample them; and (ii) all pre-existing mapper voxels inside the dynamic-obstacle workspace are removed before the fresh disks in (16)–(17) are inserted, preventing stale positions from biasing planning.

4) *Implications for the MIQP*: Because the augmentation occurs at the grid level, the MIQP formulation of FASTER is unchanged. The Whole Trajectory $\mathcal{T}_{\text{whole}}$, which is permitted to traverse $\mathcal{F} \cup \mathcal{U}$, may pass through the annular ring $\mathcal{U}_i^*$, approaching—but never entering— $\mathcal{O}_i^*$. The Safe Trajectory $\mathcal{T}_{\text{safe}}$, which is constrained to $\mathcal{F}$ only, must remain strictly outside R_i^{out} , guaranteeing clearance Δ_s beyond the committed occupancy boundary. FASTER's original guarantee—that a feasible safe braking maneuver exists at every replanning step—is therefore preserved, with the reserve Δ_s now absorbing prediction error.

C. Safety Margin Selection

The choices of $\delta_{\max}$ and Δ_s together determine the worst-case clearance $R_i^{\text{out}} = \tilde{r}_i + \delta_{\max} + \Delta_s$. Under the constant-velocity prediction and a replanning horizon $T_{\text{plan}} = N \cdot d_t$, the dominant position-prediction error is unmodeled obstacle acceleration:

$$e_{\text{pred}}(T_{\text{plan}}) \leq \frac{1}{2} a_{\text{obs,max}} T_{\text{plan}}^2. \quad (19)$$

For Safe Trajectory feasibility, the outer margin must absorb both prediction error and voxel quantization:

$$\Delta_s \geq e_{\text{pred}}(T_{\text{plan}}) + r_v. \quad (20)$$

With $\Delta_s + \delta_{\max} = 0.7$ m, $r_v = 0.15$ m, and a planning horizon $T_{\text{plan}} \approx 1$ s (set by the prediction lookahead in (11)), the outer bubble tolerates obstacles with peak acceleration of up to $a_{\text{obs,max}} \approx 1.1$ m/s². The cylinders used in our experiments move at near-constant speed with $a_{\text{obs,max}} \ll 1$ m/s², so this bound is satisfied with substantial margin.

IV. RESULTS

We evaluate our implementation in two stages. We first validate the core planning algorithms in a 2-D environment, where rapid iteration and direct visualization allow us to verify algorithmic correctness before introducing the full complexity of spatial navigation. We also implement FASTER in the 2-D environment. We compare FASTER against four baseline methods and measure three primary metrics: total time to reach the goal, total distance traveled, and number of collisions. We then implement Dynamic Faster and extend the evaluation to 3-D in both static and dynamic obstacle settings.

A. Simulation Environment

To evaluate the proposed algorithms, we implement a 2-D grid-based simulation environment with a resolution of 400×400 cells. The environment supports both static and dynamic obstacle configurations. Static maps are procedurally generated by placing circular obstacles at random positions, with a guaranteed minimum clearance between obstacles to ensure navigable paths exist. Dynamic maps extend this by assigning each obstacle a constant velocity, causing them to move and elastically bounce off the environment boundaries throughout the simulation.

The robot is modeled as a point mass with triple-integrator dynamics, where position, velocity, and acceleration evolve

under jerk control inputs. This choice of dynamics captures the smooth, continuous motion constraints relevant to aerial vehicles while remaining tractable for real-time trajectory optimization.

To simulate realistic sensing limitations, the robot is equipped with a circular perception model with a fixed radius. At each timestep, only obstacles within this radius are revealed; the remainder of the map is treated as unknown. Depending on the planning algorithm, unknown cells are either assumed free (optimistic) or occupied (conservative). For algorithms that maintain a persistent world model, an incremental known map accumulates observations over time—cells within the sensor radius are refreshed with current obstacle data at each replan step, while cells outside retain their last observed value. This is particularly relevant in dynamic environments, where an obstacle that was observed and then moved out of the sensor radius leaves behind a stale but plausible estimate of its last known position.

All algorithms share the same environment, start position, and goal across a given seed, ensuring a controlled and fair comparison.

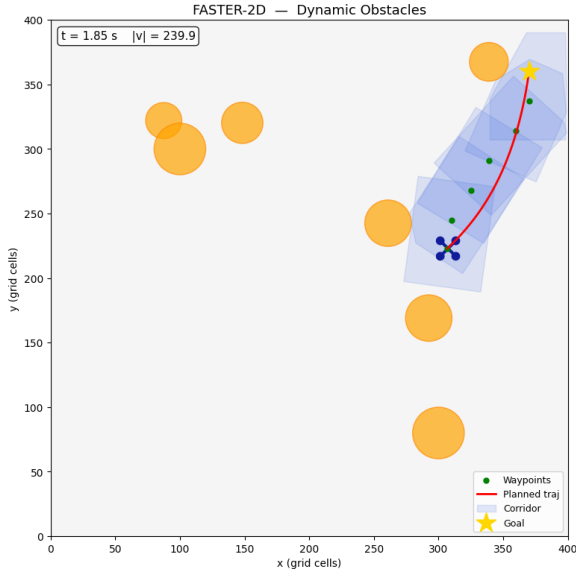


Fig. 1. 2-D Environment.

B. Baseline Methods

We compare against four baselines that span the spectrum from ideal full-knowledge conditions to realistic partial-observability planning.

1) *QP Trajectory (Full-Knowledge)*: This baseline assumes complete knowledge of the environment. A random static map is generated with circular obstacles, and A* is used to find a collision-free path from start to goal. A fixed set of waypoints is sampled along this path, and the full trajectory

is computed in a single shot using quadratic programming: the optimizer minimizes total jerk while constraining the trajectory to lie within convex free-space corridors derived from the A* path. The robot executes this trajectory without any replanning. This serves as an upper-bound baseline that isolates the quality of the trajectory optimizer under ideal, perfectly-known conditions.

2) *QP Partial-Observability*: This baseline uses A* and QP trajectory optimization under a limited sensor model, where the robot can only observe obstacles within a fixed perception radius. Unlike the optimistic and conservative RRT* variants, it does not maintain an accumulated known map—at each replan step, only the cells currently within the sensor radius are revealed, and everything outside is reset to free (unknown treated as free). This means the robot has no memory of previously observed obstacles: an obstacle seen on a past step but now outside the sensor radius is forgotten entirely. The approach is reactive and lightweight—A* replans quickly on the fresh local view, and QP generates a smooth minimum-jerk trajectory through the resulting corridors. It serves as a practical middle ground between the full-information QP baseline and the more sophisticated partial-observability methods: it handles unseen obstacles by ignoring them (optimistic), but without the long-horizon planning that accumulated maps enable.

3) *MIQP Trajectory (Full-Knowledge)*: This baseline follows the same pipeline as the QP approach—A* path, corridor decomposition, jerk minimization—but replaces the standard quadratic program with a mixed-integer quadratic program. The key difference is that binary decision variables allow the optimizer to choose, for each segment of the trajectory, which of the available convex corridors it belongs to. This gives the optimizer more flexibility when corridors overlap or when the A* path passes through ambiguous regions. Like the QP version, the full trajectory is planned once on the complete known map with no replanning.

4) *Optimistic RRT**: This baseline introduces partial observability: the robot has a limited sensor radius and can only observe cells within that range. An incremental known map is maintained, accumulating observations as the robot moves. Unknown cells—those the robot has never observed—are treated as free space. RRT* plans a path to the goal through this optimistic view of the world, potentially routing through unobserved regions. The trajectory is optimized using QP, and the robot replans periodically as new space is revealed. This approach tends to produce shorter, more direct paths but risks collision with unseen obstacles.

5) *Conservative RRT**: This baseline uses the same partial-observability setup but treats unknown cells as occupied. RRT* is therefore confined to space the robot has already confirmed as free. When the goal lies in unknown territory—as it almost always does early in the run—the planner uses frontier navigation: it identifies the farthest known-free point along the robot-to-goal direction and plans to that intermediate sub-goal instead. As the robot moves and the known map grows, the planner’s horizon extends until the goal itself

becomes reachable. This approach is safer than the optimistic variant but typically produces longer, more conservative paths and may stall when the known free region is narrow.

TABLE I
2-D STATIC ENVIRONMENT RESULTS (MEAN $\pm$ STD OVER 10 SEEDS)

Method	Distance	Time (s)	Collisions	Goal %
QP (full knowledge)	503.4 $\pm$ 16.5	2.72 $\pm$ 0.12	3.1 $\pm$ 1.9	100%
MIQP (full knowledge)	542.6 $\pm$ 94.8	1.40 $\pm$ 0.52	3.2 $\pm$ 2.6	40%
Optimistic RRT*	517.7 $\pm$ 23.6	3.05 $\pm$ 0.52	8.1 $\pm$ 7.5	100%
Conservative RRT*	507.8 $\pm$ 27.1	4.52 $\pm$ 0.21	8.9 $\pm$ 6.0	100%
FASTER	508.7 $\pm$ 26.7	2.74 $\pm$ 0.16	3.1 $\pm$ 2.4	100%

The static environment results (Table I) confirm that FASTER achieves a favorable tradeoff between speed, safety, and reliability. FASTER reaches the goal in all 10 seeds while recording a collision rate of 3.1 ± 2.4 , matching the QP full-knowledge baseline. By contrast, MIQP achieves the fastest travel time (1.40 ± 0.52 s) but succeeds in only 40% of trials due to solver failures, highlighting the brittleness of the mixed-integer formulation when applied in a single-shot open-loop setting. The two RRT* variants both reach the goal reliably but incur substantially higher collision counts (8.1 ± 7.5 for Optimistic, 8.9 ± 6.0 for Conservative), likely because their search-based paths pass closer to obstacle boundaries than the smooth corridors used by the optimization-based methods. Conservative RRT* is also the slowest method at 4.52 ± 0.21 s, reflecting the cost of restricting the planner to confirmed free space. FASTER’s flight time (2.74 ± 0.16 s) is comparable to the full-knowledge QP baseline (2.72 ± 0.12 s), demonstrating that the dual-trajectory replanning strategy recovers most of the performance available under perfect information.

6) *Dynamic Environment*: We also tested all methods in the 2-D environment with [8] dynamic obstacles moving at [30-70] grid-cells/s along linear trajectories. In this setting, the full-knowledge baselines (QP and MIQP) plan once on the initial static map and cannot react to moving obstacles, while the RRT* variants and FASTER replan but do not explicitly model obstacle motion. FASTER+Predict is the only method that incorporates predicted obstacle positions into the planning constraints.

TABLE II
2-D DYNAMIC ENVIRONMENT RESULTS (MEAN $\pm$ STD OVER 10 SEEDS)

Method	Distance	Time (s)	Collisions	Goal %
QP (full knowledge)	485.0 $\pm$ 16.9	2.58 $\pm$ 0.08	3.6 $\pm$ 5.4	50%
MIQP (full knowledge)	502.4 $\pm$ 45.8	1.31 $\pm$ 0.41	2.1 $\pm$ 3.3	90%
QP (partial-obs.)	487.6 $\pm$ 15.6	2.64 $\pm$ 0.10	2.6 $\pm$ 3.6	100%
Optimistic RRT*	496.7 $\pm$ 36.9	2.33 $\pm$ 0.23	2.5 $\pm$ 3.7	100%
Conservative RRT*	482.1 $\pm$ 11.7	3.19 $\pm$ 0.15	2.5 $\pm$ 3.4	100%
FASTER	487.8 $\pm$ 15.4	2.64 $\pm$ 0.10	1.9 $\pm$ 2.1	100%

The dynamic environment results (Table II) reveal how each method degrades when obstacles move. The most striking effect is on the full-knowledge baselines: QP drops to a 50% goal rate since its single-shot trajectory, planned on the initial obstacle configuration, becomes invalid as obstacles shift.

MIQP fares somewhat better at 90%, likely because its flexible corridor assignment produces trajectories with more clearance, but it still cannot react to changes. Among the replanning methods, FASTER achieves the lowest collision rate (1.9 ± 2.1) across all six methods while maintaining 100% goal success, suggesting that the dual-trajectory architecture—where the safe backup is recomputed at every step—provides robustness even without an explicit dynamic obstacle model. Conservative RRT* achieves the shortest distance (482.1 ± 11.7) and lowest variance, consistent with its tendency to stay close to confirmed free space, but at the cost of the longest travel time (3.19 ± 0.15 s). The QP partial-observability baseline, which replans on a memoryless local view, performs comparably to Optimistic RRT* in collision rate but with lower distance variance, indicating that the reactive memoryless strategy is a reasonable middle ground when map accumulation may introduce stale information about moved obstacles.

The 2-D results confirmed that the core FASTER replanning loop produces trajectories that are competitive with full-knowledge planners in terms of distance and time while maintaining the lowest collision rate under both static and dynamic conditions. Having validated the algorithmic pipeline in 2-D, we proceed to the full 3-D evaluation.

C. 3-D Evaluation

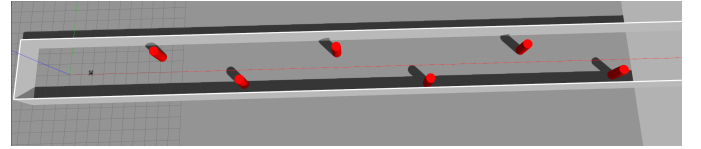


Fig. 2. 3-D Environment.

1) *Simulation Setup*: The 3-D environment (Fig. 2) is a Gazebo simulation of a 70 m $\times$ 5 m straight corridor (“Moving Wall Hall”), bounded on both sides by static walls of height 3 m. Eight cylindrical dynamic obstacles of diameter 0.8 m and height 2.5 m are distributed evenly along the corridor at $x \approx \{8, 16, 24, 32, 40, 48, 56, 64\}$ m with small random jitter. Each cylinder oscillates sinusoidally along the y -axis across the corridor width at a random speed drawn uniformly from $[0.5, 1.0]$ m/s; amplitudes and phases are also randomized so the cylinders do not share a common rhythm.

The drone is modeled as a quadrotor carrying an Asus Xtion Pro depth camera (90° horizontal FoV, 60 Hz, 320×240 resolution, 7 m maximum range). Its dynamics follow the triple-integrator model of Section II-A, with $v_{\max} = 5$ m/s, $a_{\max} = 5$ m/s², and $j_{\max} = 8$ m/s³. The `global_mapper` node fuses incoming point clouds into a 0.15 m voxel grid published at 20 Hz, and FASTER replans at 100 Hz. The drone spawns at (2, 0, 0.3) m with goal (68, 0, 1) m, requiring it to traverse the full corridor while threading through the eight moving cylinders. Physics is enabled throughout; any drone-cylinder overlap is logged as a collision.

2) *Dynamic Environments*: To evaluate FASTER+Predict, we introduce $M = 8$ moving cylindrical obstacles into the Moving Wall Hall. Each cylinder (0.8 m diameter, 2.5 m height) moves sinusoidally along the y -axis at a randomized speed in $[0.5, 1.0]$ m/s, with randomized amplitude and phase. The inflation parameters used throughout are $K = 0.2$, $\delta_{\min} = 0.05$ m, $\delta_{\max} = 0.3$ m, $\Delta_s = 0.4$ m. We conducted nine independent trials each for vanilla FASTER and FASTER+Predict, randomizing cylinder phase between runs. Fig. 3– 7 report the resulting metrics.

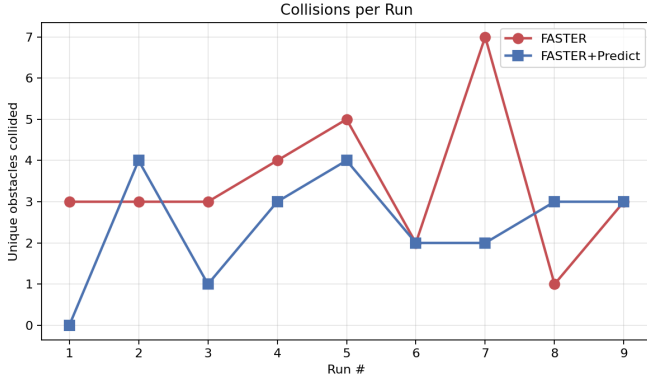


Fig. 3. Collisions.

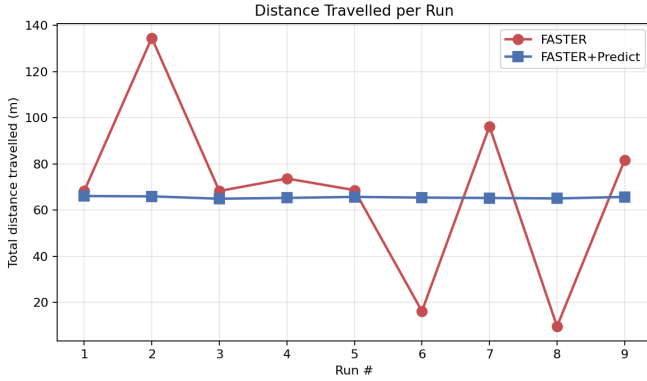


Fig. 4. Distance Travelled.

a) *Collisions (Fig. 3)*: Fig. 3 reports the number of unique cylinders each method collided with per run. Vanilla FASTER has no knowledge of obstacle motion: the committed trajectory is computed on a snapshot of the occupancy grid that is already stale by the time the drone reaches it, so collisions depend on chance alignment of drone and cylinder phases. FASTER+Predict reduces the collision count in nearly every trial, the inflated bubble around each predicted obstacle position gives the planner a consistent 1 s lookahead margin, which prevents committing to a path that will intersect a cylinder’s near-future position. Residual collisions are concentrated in runs where a cylinder passes close to the corridor wall, leaving only a narrow passable gap.

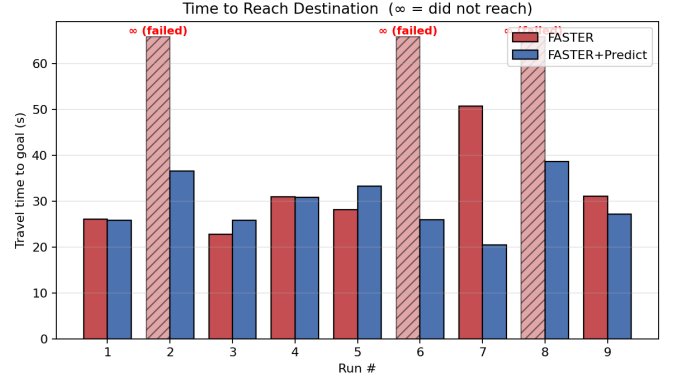


Fig. 5. Travel Time.

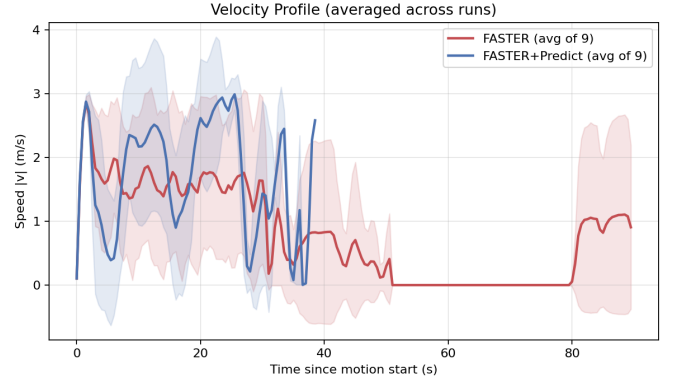


Fig. 6. Velocity Profile.

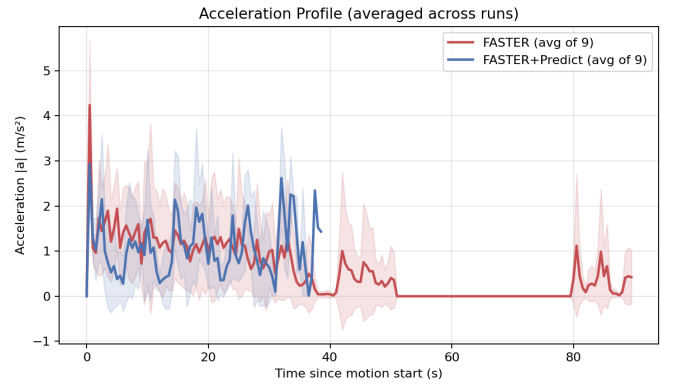


Fig. 7. Acceleration Profile.

b) *Distance (Fig. 4)*: The total distance travelled is modestly larger but constant for FASTER+Predict than for vanilla FASTER. This is expected: the inflated bubbles push the planned trajectory farther from each cylinder’s predicted position, producing more lateral displacement as the drone weaves around obstacles. The straight-line goal distance is 66 m; both methods travel close to this value, confirming that the inflation model is not overly conservative, it adds only the lateral excursions needed to clear the dynamic obstacles.

c) *Travel time (Fig. 5)*: Fig. 5 shows per-trial time to reach the goal, with unreached trials displayed as ∞ . Vanilla FASTER exhibits a bimodal distribution: successful trials complete in a similar time to the predict variant, but a non-trivial fraction fail completely—the drone stalls behind a cylinder or the safe trajectory becomes infeasible. FASTER+Predict eliminates most of these failures. The reduction in failure rate is the more significant effect: by anticipating the swept region of each cylinder, the planner avoids the stall-and-wait behavior that arises when vanilla FASTER finds itself moving parallel to an approaching obstacle with no clear gap.

d) *Velocity profile (Fig. 6)*: Fig. 6 plots the speed $\|\dot{\mathbf{q}}_d\|$ averaged across the nine trials per method, versus time since motion onset. Both methods accelerate toward $v_{\max} = 5$ m/s in the first ~ 1 s, then modulate their speed in response to obstacle encounters. Vanilla FASTER shows sharper peaks and a wider shaded $\pm\sigma$ band, reflecting abrupt braking maneuvers when an obstacle emerges into its path. FASTER+Predict produces a smoother profile: because predicted obstacle positions are already baked into the planning grid, the drone does not need to execute sudden slowdowns. The mean cruise speed is comparable, but the smoother profile is consistent with the hypothesis that predictive inflation reduces reactive braking.

e) *Acceleration profile (Fig. 7)*: The acceleration profile reinforces the velocity-smoothing observation. Vanilla FASTER operates at larger acceleration magnitudes throughout the run—particularly in the middle third where encounter density is highest—with peaks approaching $a_{\max} = 5$ m/s². FASTER+Predict operates at consistently lower acceleration magnitudes, indicating that the planner spends less time at the kinodynamic limit. This is an indirect measure of actuator comfort and tracking accuracy: a drone that plans ahead does not have to expend peak control authority to avoid obstacles at the last moment.

f) *Summary*: Across all five metrics, FASTER+Predict provides measurable improvements on the dynamic benchmark: fewer collisions, higher goal success rate, smoother velocity and acceleration profiles, at the cost of a small increase in path length. The results validate the central hypothesis of the dynamic extension—that per-cycle prediction of obstacle positions, combined with a two-boundary inflation scheme that integrates cleanly with FASTER’s existing MIQP, is sufficient to restore reliable performance in environments where the static-world assumption of the original formulation breaks down.

D. Technical Challenges and Limitations

Several practical difficulties arose during this project. Most notably, the original FASTER paper [1] benchmarked against seven baseline methods and reported results on an Intel Core i7-7700HQ processor with Gurobi as the MIQP solver. Due to limited computational resources, we were unable to fully reproduce the original paper’s seven-method comparison. Instead, we designed our own set of four baselines that span the spectrum from full-knowledge planning (QP, MIQP) to partial-observability planning (Optimistic and Conservative RRT*), providing a meaningful basis for evaluating FASTER and our dynamic extension within our available compute budget.

A further simplification concerns the perception pipeline for the FASTER+Predict experiments. In principle, the obstacle centroids and velocities used in (11) should be extracted by clustering the depth-camera point cloud and tracking clusters across frames. In practice, the stock Gazebo occupancy mapper publishes its voxel grid at only 20 Hz and decays stale voxels slowly (-0.01 per miss), producing persistent trails behind moving obstacles that corrupted the cluster centroids and biased the velocity estimates. To isolate the contribution of the two-boundary inflation model from these unrelated artifacts of the mapping pipeline, we source obstacle positions directly from the simulator’s model-state channel, feed them through the same velocity-estimation filter (11) that a real tracker would implement, and then apply the two-boundary inflation identically. This substitution affects only the *input* to the planner; the MIQP, corridor construction, and safe-trajectory logic remain unchanged. Integrating the method with a production cluster tracker (e.g. Euclidean clustering with a Kalman filter per cluster) is a direct extension of this work.

V. CONCLUSION

This work revisited the FASTER trajectory planner and examined its behavior beyond the static-world assumption under which it was originally designed. We reproduced the core algorithm in both 2-D (Python) and 3-D (ROS/Gazebo) simulations, confirming that its dual-trajectory architecture—a *whole* trajectory optimized through free-known and unknown space, paired with a guaranteed *safe* trajectory confined to free-known space—recovers most of the performance available under full environment knowledge while retaining formal safety guarantees.

We then proposed *Dynamic FASTER*, a minimal extension that handles moving obstacles without modifying the MIQP, the convex-corridor construction, or the committed-trajectory mechanism. The extension operates entirely at the perception layer: tracked obstacles are inflated by a distance- and velocity-adaptive inner bubble published as occupancy, surrounded by a fixed safety annulus published as unknown space. This two-boundary grid augmentation maps directly onto the inputs that the original FASTER pipeline already consumes, so the whole/safe guarantees carry over unchanged.

Experiments in a 70 m corridor populated with eight sinusoidally moving cylinders show that Dynamic FASTER measurably reduces collision counts, increases goal-reach success

rate, and produces smoother velocity and acceleration profiles than the vanilla planner, at a small cost in path length. These results support our central hypothesis: reliability in dynamic environments can be restored by augmenting perception rather than by complicating the optimizer.

That said, the proposed method does not eliminate collisions entirely. Residual failures occur primarily when a cylinder is close to the corridor wall and leaves only a narrow passable gap, or when its turnaround-point acceleration briefly exceeds the worst-case bound assumed in our margin derivation (19). In such cases, the drone enters a corridor that the predictor expected to be free by the time the obstacle reversed direction. The inflation parameters K , $\delta_{\max}$, and Δ_s governing bubble size present a direct trade-off between safety and navigability: larger bubbles reduce collisions but can choke the corridor and cause the safe trajectory to become infeasible; smaller bubbles improve throughput but allow tighter encounters. A more thorough sensitivity study across these parameters—ideally combined with an adaptive scheme that inflates bubbles based on measured prediction error—is required before the approach can be considered production-ready.

Further directions include driving the velocity estimator from a real point-cloud clustering and tracking pipeline rather than the simulator’s model-state channel used here, and validating the method on hardware, where mapper latency and sensor noise will test the robustness of the two-boundary inflation model beyond what a simulated environment can reveal.

REFERENCES

- [1] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, “FASTER: Fast and safe trajectory planner for navigation in unknown environments,” *IEEE Trans. Robot.*, vol. 38, no. 2, pp. 922–938, Apr. 2022.
- [2] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhatt, C. J. Taylor, and V. Kumar, “Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments,” *IEEE Robot. Autom. Lett.*, vol. 2, no. 3, pp. 1688–1695, Jul. 2017.
- [3] C. Richter, A. Bry, and N. Roy, “Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments,” in *Proc. Int. Symp. Robot. Research (ISRR)*, 2016, pp. 649–666.
- [4] J. Tordesillas, B. T. Lopez, J. Carter, J. Ware, and J. P. How, “Real-time planning with multi-fidelity models for agile flights in unknown environments,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 725–731.
- [5] D. Harabor and A. Grastien, “Online graph pruning for pathfinding on grid maps,” in *Proc. AAAI Conf. Artif. Intell.*, 2011, pp. 1114–1119.
- [6] B. Luders, S. Karaman, and J. P. How, “Robust sampling-based motion planning with asymptotic optimality guarantees,” in *Proc. AIAA Guid., Navig., Control Conf.*, 2013.
- [7] P. Hart, N. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.